

Advanced MySQL Self-Study Manual — Part 3: Interview Prep, Traps, Plan, Capstone

PART 27 — INTERVIEW QUESTION BANK

Organized by category. Each entry: question → expected concept → model answer → common wrong answer → likely follow-up.

Basic

1. **What is the difference between WHERE and HAVING?** Concept: filter timing relative to grouping. Model answer: WHERE filters individual rows before grouping; HAVING filters groups after aggregation. Common wrong answer: “They’re interchangeable” — fails the moment an aggregate function is involved. Follow-up: “Can you use HAVING without GROUP BY?” (Yes — it then filters against the whole result set as one group.)
2. **What’s the difference between COUNT(*) and COUNT(column)?** Model answer: COUNT(*) counts all rows regardless of NULLs; COUNT(column) counts only non-NULL values in that column. Follow-up: “What does COUNT(1) do?” (Same as COUNT(*) in MySQL — a common myth is that it’s faster; it isn’t.)
3. **What’s the difference between DELETE, TRUNCATE, and DROP?** Model answer: DELETE removes rows (can be filtered, is transactional, fires triggers, slower); TRUNCATE removes all rows (resets auto-increment, minimal logging, can’t be filtered, generally not rollback-able the same way); DROP removes the entire table structure. Follow-up: “Can you ROLLBACK a TRUNCATE in InnoDB?” (Historically no reliably across engines; behavior nuance worth knowing but engine/version-dependent — flag it as “verify for your exact MySQL version” rather than asserting confidently either way.)
4. **What’s the difference between a PRIMARY KEY and a UNIQUE constraint?** Model answer: a table can have only one primary key (which also implies NOT NULL); a table can have multiple unique constraints, and unique columns can allow one NULL (though behavior with multiple NULLs varies by database — MySQL allows multiple NULLs in a unique column).
5. **What is a foreign key and why use one?** Model answer: enforces referential integrity — a value in the child table must exist in the parent table (or be NULL), preventing orphaned records.

Intermediate

6. **Find customers who have never placed an order.** → anti-join pattern (LEFT JOIN...IS NULL or NOT EXISTS).
7. **What’s the difference between EXISTS and IN?** → EXISTS checks row existence (stops at first match, correlated), IN checks list membership (evaluates the full list/subquery); EXISTS is safer with NULLs.
8. **Why might NOT IN return unexpected empty results?** → NULL

in the subquery result poisons the whole comparison to UNKNOWN.

9. **What's a self join and when would you use one?** → joining a table to itself, e.g., employee-to-manager, or finding duplicate/related rows.
10. **Explain GROUP BY vs a window function conceptually.** → GROUP BY collapses rows; window functions keep every row and add a computed column.

Advanced

11. **Find the second-highest salary in every department.** → `DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC)`, filter rank = 2.
12. **What is a correlated subquery and how does it differ in execution from a non-correlated one?** → covered fully in Part 1, Part 4.
13. **Explain ROWS vs RANGE in a window frame.** → physical row count vs. logical peer-group range; matters with duplicate ORDER BY values.
14. **What is a recursive CTE and what are its two required parts?** → anchor + recursive member joined with UNION ALL, with a termination condition.
15. **How would you find the longest streak of consecutive days a customer placed an order?** → gaps-and-islands: subtract a row-number sequence from the date to get a constant "island key" per streak.

Senior

16. **Walk me through how you'd design the schema for an e-commerce order system from scratch.** → expects discussion of normalization, entity relationships (customers/orders/order_items/products), handling of price-at-time-of-order (don't just FK to current product price — snapshot it), indexing strategy for common queries.
17. **A report query joins 6 tables and takes 30 seconds. How do you approach optimizing it?** → expects: run EXPLAIN/EXPLAIN ANALYZE, check for full scans, check index coverage on join/filter columns, check join order, check if you're pulling unnecessary columns, check if aggregation could be pre-computed/materialized, check the pagination approach.
18. **How would you migrate a large table's schema (e.g., add a NOT NULL column) with zero downtime?** → expects awareness of online DDL, adding nullable first + backfilling + adding constraint after, or tools like pt-online-schema-change/gh-ost for very large tables.
19. **Explain how you'd detect and prevent a recurring deadlock in production.** → expects: consistent lock ordering across transactions, checking SHOW ENGINE INNODB STATUS for deadlock details, keeping transactions short, indexing to reduce lock scope.
20. **When would you choose denormalization?** → read-heavy reporting/analytics workloads where join cost outweighs write-time duplication cost; OLAP vs OLTP tradeoff discussion.

Performance

21. **What does type = ALL mean in EXPLAIN?** → full table scan; MySQL examines every row.
22. **What is a covering index?** → an index containing every column the query needs, so MySQL never touches the base table row.
23. **Why can SELECT * hurt performance even beyond "extra columns"?** → can prevent a covering index from applying, forcing an extra row lookup per matched index entry.

24. **What is the leftmost-prefix rule for composite indexes?** → a composite index can serve queries filtering on a left-aligned subset of its columns, in order; it can't efficiently serve a query filtering only on a non-leading column.
25. **Explain keyset (seek) pagination vs offset pagination and why one scales better.** → offset must scan-and-discard prior rows (cost grows with page depth); keyset jumps directly via an indexed `WHERE column > last_seen_value`.

Database Design

26. **Explain 1NF, 2NF, 3NF with an example of a violation for each.**
27. **What's the difference between OLTP and OLAP systems?**
28. **When would you intentionally violate normalization?**
29. **What's a surrogate key vs a natural key, and which would you generally prefer for a primary key?**
30. **How would you model a many-to-many relationship (e.g., students and courses)?** → junction/bridge table.

Transactions

31. **Explain ACID with a concrete banking example.**
32. **What's the difference between READ COMMITTED and REPEATABLE READ?**
33. **What is a phantom read, and does MySQL's REPEATABLE READ prevent it?** → largely, via MVCC for plain reads, though `SELECT...FOR UPDATE` can still see phantoms in some cases.
34. **What's a SAVEPOINT and when would you use one?**
35. **What causes a deadlock, and how does InnoDB handle it when one occurs?**

MySQL Internals

36. **What is a clustered index, and how does InnoDB use the primary key for it?**
37. **What is the buffer pool and why does its size matter for performance?**
38. **Why might MySQL's optimizer ignore an index that "should" help?** → stale statistics, low selectivity, small table where a full scan is genuinely cheaper.
39. **What's the difference between a stored procedure and a stored function?**
40. **What's a generated column, and when would you make one STORED vs virtual (computed on read)?**

PART 28 — SQL PATTERN CHEAT SHEET (quick reference)

Latest record per group <code>ORDER BY date DESC, rn = 1</code>	→ <code>ROW_NUMBER() PARTITION BY group</code>
Top N per group	→ <code>ROW_NUMBER() ... rn <= N</code>
Second/Nth highest value <code>rnk = N</code>	→ <code>DENSE_RANK() ORDER BY value DESC,</code>
Customers without orders <code>EXISTS</code>	→ <code>LEFT JOIN ... IS NULL or NOT</code>
Customers with ≥1 order	→ <code>EXISTS (semi-join)</code>
Duplicate records	→ <code>GROUP BY key HAVING COUNT(*) > 1</code>
Deduplicate, keep one <code>ORDER BY tiebreak, rn = 1</code>	→ <code>ROW_NUMBER() PARTITION BY key</code>

Running total	→ SUM(x) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
Moving average	→ AVG(x) OVER (ORDER BY date ROWS BETWEEN n PRECEDING AND CURRENT ROW)
Previous / next row	→ LAG() / LEAD()
Month-over-month change	→ value - LAG(value) OVER (ORDER BY period)
Percentage of total	→ x / SUM(x) OVER ()
Gaps and islands (streaks)	→ date - ROW_NUMBER() as island key, GROUP BY island key
Hierarchy / org chart	→ WITH RECURSIVE, anchor + UNION
ALL + recursive join	
Relational division	→ GROUP BY + HAVING COUNT(DISTINCT x) = (subquery total count)
Compare two datasets	→ anti-joins both directions + inner join for "changed" rows
Correlated vs global aggregate	→ does the comparison value change per row/group? yes=correlated, no=non-correlated
Non-sargable date filter fix	→ col >= 'start' AND col < 'end', never wrap col in a function

PART 29 — INTERVIEW TRAPS

- **WHERE vs HAVING** — WHERE can't reference aggregates; HAVING runs after grouping.
 - **WHERE vs ON** — a right-table condition in WHERE turns a LEFT JOIN into an effective INNER JOIN; keep it in ON to preserve unmatched rows.
 - **INNER JOIN vs LEFT JOIN** — silent row loss if you meant to keep unmatched rows.
 - **COUNT(*) vs COUNT(column)** — NULLs are skipped by the latter.
 - **NULL comparisons** — = NULL never matches; must use IS NULL.
 - **NOT IN with NULL** — poisons the whole condition to UNKNOWN, returns zero rows.
 - **RANK() vs DENSE_RANK()** — gap after ties vs no gap.
 - **ROW_NUMBER() vs RANK()** — always-unique vs ties-share-a-value.
 - **GROUP BY vs window function** — collapses rows vs. keeps every row.
 - **UNION vs UNION ALL** — dedup cost vs. none; default to UNION ALL unless you need dedup.
 - **EXISTS vs IN** — NULL-safety and correlation differences.
 - **DELETE vs TRUNCATE vs DROP** — filtered/transactional vs. reset-all vs. remove-structure.
 - **PRIMARY KEY vs UNIQUE** — one per table + implies NOT NULL, vs. many allowed + NULLs permitted.
 - **Function wrapping an indexed column** — breaks sargability, forces a scan.
 - **Composite index column order** — leftmost-prefix rule; wrong order makes the index useless for your actual filter.
 - **Accidental LEFT JOIN → INNER JOIN** — condition misplaced in WHERE instead of ON.
 - **Duplicate rows after a JOIN** — joining to a table where the join key isn't unique multiplies rows ("fan-out") — always sanity-check row counts after adding a join.
 - **Correlated subquery performance** — recomputed per row conceptually; can be far slower than a join/window equivalent at scale.
 - **CTE misconceptions** — a CTE is not automatically materialized or indexed; MySQL's optimizer decides whether to inline or materialize it.
 - **LAST_VALUE window-frame trap** — default frame returns the current row, not the true last row, unless you widen it explicitly.
-

PART 30 — FINAL CAPSTONE: E-Commerce + Compensation Analytics

Using the advanced_sql_lab database from Part 2 (extend it with more rows if you want realism at scale), solve these business requirements. This is meant to feel like a real work ticket queue, not a quiz.

1. Produce total revenue by month for the last 12 months of order data.
2. Rank customers by lifetime spend and flag the top 10%.
3. Identify the top 3 best-selling products by quantity, per category.
4. For each customer, find their most recent completed order and how many days it's been since.
5. Detect duplicate customer records (same name, different customer_id) — decide and document your matching logic.
6. Find products with no sales in the last 90 days (if simulating "now," pick a fixed reference date).
7. Reconcile order_items totals against payments.amount; report mismatches.
8. Compute month-over-month revenue growth rate as a percentage.
9. Compute a running total of company revenue, ordered by date.
10. Compute customer retention: of customers who ordered in month N, what % also ordered in month N+1?
11. Build the full employee reporting hierarchy with each employee's depth level from the top.
12. Compare each employee's current salary against their department's average, flagging outliers > 20% above.
13. Produce an audit trail: for every salary change, show old salary, new salary, and % change, most recent first per employee.
14. Take the slowest query you wrote above, run EXPLAIN, and propose (and create) at least one index to improve it — document the before/after EXPLAIN output.

Evaluate your own solutions against: correctness (does it handle NULLs/edge cases in the seed data?), query design (readable, uses the right pattern), index design (did you index what you filter/join/sort on?), performance (did you check EXPLAIN?), maintainability (would a teammate understand this in 6 months?), and edge cases (ties, orphans, NULLs — all deliberately seeded in Part 2's data).

PART 31 — MOCK INTERVIEW STRUCTURE (self-administered)

Run this as a timed session. Write your answers before checking Parts 1-2. Score yourself 1-10 per round; anything below 7 is a signal to revisit that Part.

Round	Focus	Reference
1	SQL fundamentals	Part 1
2	JOINS	Part 3 (JOIN masterclass)
3	Subqueries	Part 4
4	CTEs	Part 5
5	Window functions	Part 6
6	Advanced aggregation	Part 7
7	Optimization	Part 11
8	Database design	Part 12
	Transactions &	

9	locking	Parts 13-15
10	Real-world problem	Part 30 capstone item of your choice
11	Senior architecture discussion	“Design a database for X” — pick any domain and talk through it out loud

PART 32 — 30/60/90 DAY PLAN

Days 1-30 — Core advanced SQL - Week 1: JOINS (all variants) + subqueries — Parts 3-4, Level 1-2 exercises. - Week 2: CTEs + recursive CTEs — Part 5, Level 3 exercises #24, #29. - Week 3: Window functions (the biggest section — don’t rush it) — Part 6, Level 3 exercises #21-23, #25-28. - Week 4: Advanced aggregation + full pattern library review — Parts 7-8, Level 4 exercises.

Days 31-60 — Optimization, internals, transactions - Week 5: Indexing, EXPLAIN, sargability — Part 11, Optimization Lab (Part 26). - Week 6: Normalization + database design — Part 12; sketch a schema for a domain of your choice from scratch. - Week 7: Transactions, isolation levels, locks/deadlocks — Parts 13-15. - Week 8: Views, stored procedures, triggers, MySQL-specific features — Parts 16-19.

Days 61-90 — Interview readiness + production thinking - Week 9: Full interview question bank (Part 27), timed — no notes. - Week 10: Debugging lab + optimization lab, redone from memory (Parts 25-26). - Week 11: Capstone project (Part 30), all 14 requirements. - Week 12: Full mock interview (Part 31) end-to-end, then revisit your weakest 2-3 topics.

PART 33 — PROGRESS CHECKLIST

- [] Advanced JOINS (inner/left/right/full/self/cross/semi/anti)
- [] Non-correlated subqueries
- [] Correlated subqueries
- [] EXISTS / NOT EXISTS
- [] IN / NOT IN (+ NULL trap)
- [] Basic CTEs
- [] Recursive CTEs
- [] ROW_NUMBER / RANK / DENSE_RANK
- [] LAG / LEAD
- [] FIRST_VALUE / LAST_VALUE (+ frame trap)
- [] Window frames (ROWS vs RANGE)
- [] Conditional aggregation
- [] ROLLUP
- [] Gaps and islands
- [] Deduplication
- [] Reconciliation pattern
- [] Relational division
- [] Indexes (primary/unique/secondary/covering)
- [] Composite indexes / leftmost-prefix
- [] EXPLAIN / EXPLAIN ANALYZE
- [] Sargability
- [] Pagination (offset vs keyset)
- [] Normalization (1NF-BCNF)
- [] OLTP vs OLAP
- [] Transactions / ACID
- [] Isolation levels
- [] Locks / deadlocks

- [] Views
- [] Stored procedures / functions
- [] Triggers
- [] MySQL internals (B-tree, buffer pool, optimizer)
- [] JSON / generated columns / full-text search
- [] Capstone project completed
- [] Full mock interview completed

(This completes the manual. Part 1: concepts & patterns. Part 2: practice database, exercises, answers, debugging & optimization labs. Part 3: interview bank, cheat sheets, traps, capstone, study plan.)